RSCL — Algorithm 8: Swarm (Rigid-Group) Travel in the Arena Frame

Reconfigurable Space Computing Lab (RSCL), California State Polytechnic University, Pomona (Cal Poly Pomona) · Director: Prof. Mohamed El-Hadedy

Date: 2026-11-12 · Document type: formal algorithm specification of the implemented code (HU-MA'AT drone-fleet simulation demo)
Source files described (content hash and state at document build time):

hu-telepresence-gazebo/festival/control_server_hu.py	sha256 3f32578a144c	2026-11-11 06:32 UTC	clean (HEAD 983eede; last change 10ee643)
hu-telepresence-gazebo/festival/mission_intents.py	sha256 e109c5a9e89c	2026-11-12 01:11 UTC	modified vs b44bc1c

1. Purpose and context

An operator command such as "All move forward 1 meter" or "Horus, Ra and Thoth go back 40 centimeters" moves a group as a rigid body. Because all members receive an identical translation, their mutual distances are unchanged in the ideal model, so the formation keeps its shape and intra-group conflicts cannot arise from the plan itself. Direction words are interpreted in the **arena frame** (the yaw-0 heading), not in each aircraft's body frame, so that aircraft with different headings still move in parallel. (A single named aircraft is moved in its body frame instead, Algorithm 7.2.)

2. Inputs and outputs

Input: targets (|targets| > 1, from Algorithm 3 with operator confirmation), parameters among `dx, dy, dz, forward_m, right_m` (metres). **Output:** goal map {name: (x, y, z)}; the supervisor sets each target and mode MOVE; arrival returns each member to HOVER (Algorithm 9).

3. Algorithm

Algorithm 8.1 PLAN-GROUP-MOVE

Source: festival/control_server_hu.py: Fleet._plan_group_move (caller Fleet.command holds the lock)

```
function PLAN-GROUP-MOVE(targets, params)
  read dx, dy, dz, fwd, right (default 0); require all finite and each |·| ≤ 2.0
  Δ ← (dx + fwd,  dy − right,  dz)                       // arena ENU: forward = +x, right = −y
  M ← members;  O ← fleet \ M
  for d in M do
    require not d.fault
    require d.mode ∈ {HOVER, FORMATION}
    require ‖p_d − target_d‖ ≤ 0.12 and ‖v_d‖ ≤ 0.12     // settled
  for o in O do
    require o.mode ∉ {MOVE, FORMATION, TAKEOFF, LANDING, SPIN, TURN}
    require fresh(o)
  for each pair a, b in M do require ‖p_a − p_b‖ ≥ 0.88
  for d in M do
    g_d ← p_d + Δ
    require |g_d.x|, |g_d.y| ≤ 3.35 and 0.65 ≤ g_d.z ≤ 2.8          // refused, never clamped
    for o in O, for k = 0..50 do
      if ‖p_d + Δ·k/50 − p_o‖ < 1.0 then refuse "would pass within 1.0 m of o"
  return { d: g_d }
```

Members' freshness is checked earlier in `Fleet.command` (risk-increasing actions with stale targets fail closed), and the MOVE-family precondition requires each member to be airborne.

4. Flowchart and state diagram

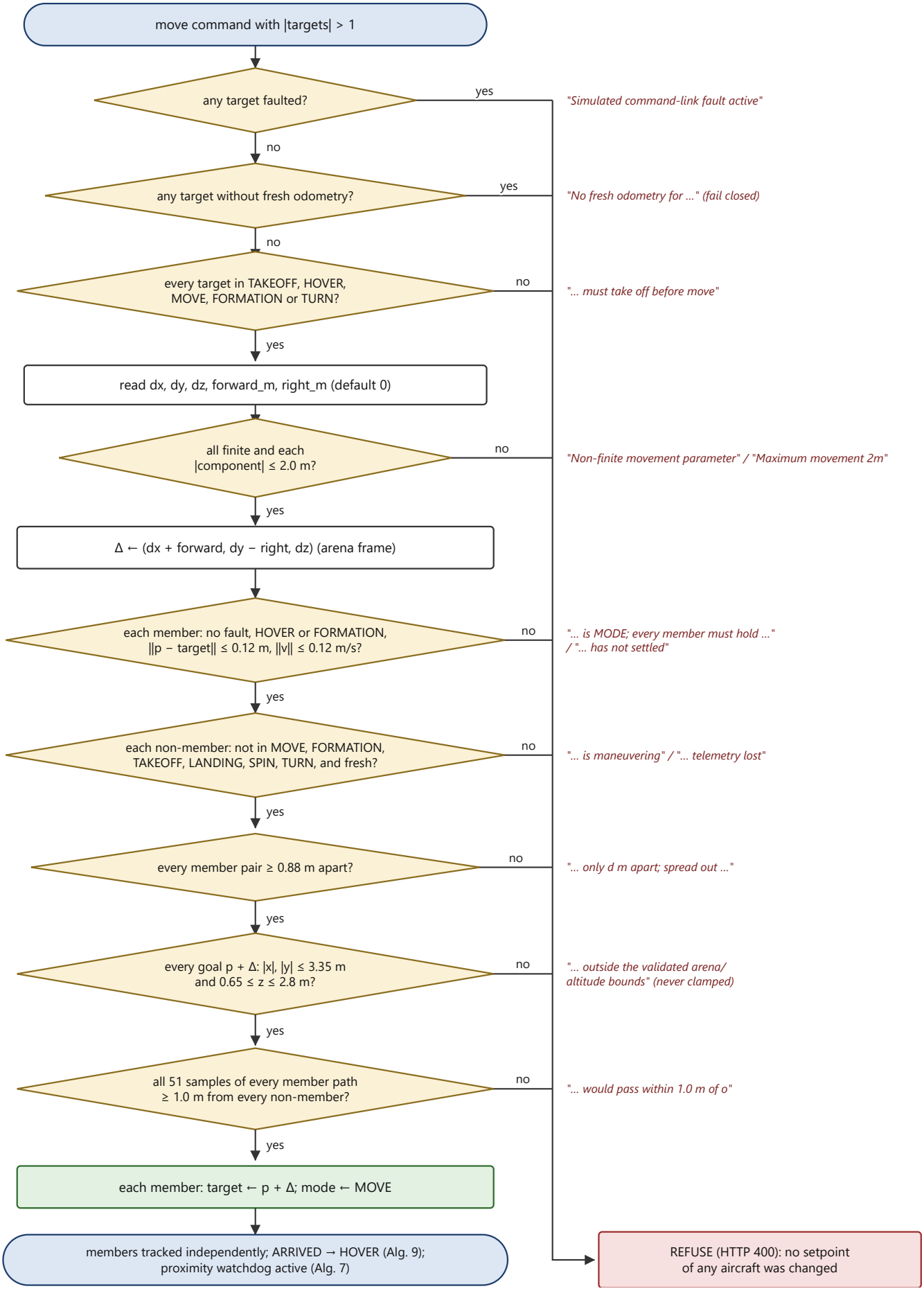

move command with |targets| > 1
any target faulted?
yes
"Simulated command-link fault active"
no
any target without fresh odometry?
yes
"No fresh odometry for ..." (fail closed)
no
every target in TAKEOFF, HOVER, MOVE, FORMATION or TURN?
no
"... must take off before move"
yes
read dx, dy, dz, forward_m, right_m (default 0)
all finite and each |component| ≤ 2.0 m?
no
"Non-finite movement parameter" / "Maximum movement 2m"
yes
Δ ← (dx + forward, dy − right, dz) (arena frame)
each member: no fault, HOVER or FORMATION, ||p − target|| ≤ 0.12 m, ||v|| ≤ 0.12 m/s?
no
"... is MODE; every member must hold ..." / "... has not settled"
yes
each non-member: not in MOVE, FORMATION, TAKEOFF, LANDING, SPIN, TURN, and fresh?
no
"... is maneuvering" / "... telemetry lost"
yes
every member pair ≥ 0.88 m apart?
no
"... only d m apart; spread out ..."
yes
every goal p + Δ: |x|, |y| ≤ 3.35 m and 0.65 ≤ z ≤ 2.8 m?
no
"... outside the validated arena/ altitude bounds" (never clamped)
yes
all 51 samples of every member path ≥ 1.0 m from every non-member?
no
"... would pass within 1.0 m of o"
yes
each member: target ← p + Δ; mode ← MOVE
members tracked independently; ARRIVED → HOVER (Alg. 9); proximity watchdog active (Alg. 7)
REFUSE (HTTP 400): no setpoint of any aircraft was changed

Figure 1. Admission flowchart of a group move: the first three checks are in `Fleet.command`, the rest in `Fleet._plan_group_move`. Every refusal raises `ValueError` (HTTP 400) before any setpoint is written; the italic text is the refusal message.

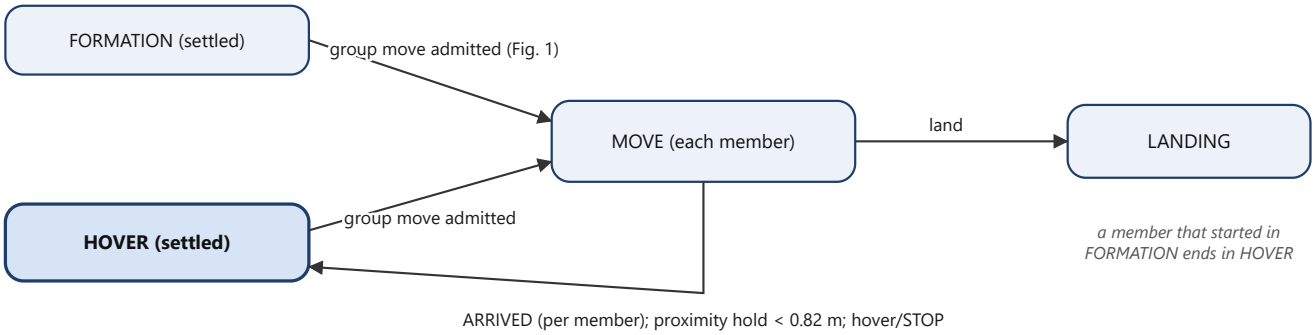


Figure 2. Mode transitions of each member during a group move (subset of the supervisor FSM, Algorithm 9). Members arrive individually; the group has no state of its own.

5. Parameters

Parameter	Value
Max component per command	2.0 m (mission parser further restricts a spoken move to 0.10..2.0 m in one axis)
Settled tolerance	position-to-setpoint ≤ 0.12 m and speed ≤ 0.12 m/s
Minimum intra-group spacing	0.88 m (MIN_SEP)
Clearance to non-members	1.0 m at 51 samples per member path
Bounds	$\|x\|, \|y\| \le 3.35$ m; $0.65 \le z \le 2.8$ m

6. Complexity

$O(|M|^2 + 51 \cdot |M| \cdot |O|)$; at most $51 \cdot 6 = 306$ distance checks for $N = 5$.

7. Properties

Guaranteed

- All members get the identical displacement; planned pairwise distances inside the group are unchanged (shape preserved in the setpoint model).
- No partial or clamped execution: any violated condition refuses the whole group before any setpoint changes.
- Non-members must be stationary (not in a manoeuvring or landing mode) and fresh; each member's straight path is sampled against them at 1.0 m.
- Arena-frame semantics: "right" is −y for every member regardless of its yaw.

Not guaranteed

- The controller tracks each member independently (per-axis velocity limits, PD law), so members do not move in lock-step; intra-group spacing can vary transiently. Only the watchdog (0.82 m) protects during the move.
- Members start from their current positions, not their nominal slots; small residual slot errors carry over.
- Clearance to non-members is sampled (1/50 of the leg) and assumes they remain where they are.
- The arena-frame convention differs from the body-frame convention for a single aircraft; the operator must know whether one or several aircraft are addressed.
- A member that started in FORMATION ends the move in HOVER (MOVE always completes to HOVER), so a later step that expects FORMATION mode must re-issue the formation.

8. Testing and measured results

- Offline: `acceptance/tests/test_v9.py`: `test_swarm_move_is_rigid_translation`, `test_swarm_right_is_arena_minus_y`, `test_swarm_refuses_grounded_member_without_mutation`, `test_swarm_refuses_unsettled_member`, `test_swarm_refuses_path_near_stationary_aircraft`, `test_swarm_refuses_out_of_bounds_without_clamping`, `test_swarm_refuses_faulted_member`.
- Live, `acceptance/swarm_test.py` (pass criteria: each member moved by the commanded vector ±8 cm, every pairwise distance changed < 6 cm, members back in HOVER, non-members did not move). Run `run_20261111T052914Z`: 7/7 legs PASS (forward 1 m, right 50 cm, back 1 m, left 50 cm, up 30 cm, down 30 cm, sub-group back 40 cm with Anubis holding); settle 2.13–3.64 s; shape change ≤ 0.012 m; e.g. "forward 1 m" moved Horus [0.96, −0.01, 0.0] m; minimum observed separation 1.105 m (Horus–Anubis); no non-member moved.

9. Comparison with alternatives and related work

Formation control is commonly classified as virtual-structure, leader–follower or consensus/behaviour-based, or, by what agents sense, as position-, displacement- or distance-based [4]. The HU-MA'AT group move is a position-based, open-loop instance of the virtual-structure idea: a rigid displacement is applied once to all setpoints in the global (arena) frame, and each aircraft then tracks its own setpoint with no feedback from the others.

Approach	Mechanism	What it guarantees (under its assumptions)	Compared with ours
Virtual structure [1]	The formation is a rigid body; the structure is moved, each robot tracks its point in it, and robot errors are fed back to the motion of the structure.	High-precision shape keeping; with feedback the structure slows down when a robot lags.	Ours applies the rigid displacement once and has no feedback from members to the structure: a lagging member is not waited for.
Leader–follower [2]	Followers control distance and bearing (or two distances) to a leader or neighbours along a control graph.	Stability of the shape variables for the modelled (nonholonomic) robots; needs only relative sensing.	Ours needs global positions from the simulator and has no leader; a single aircraft's failure does not propagate, but it is also not compensated.
Consensus-based [3]	Each agent updates its state from neighbours over a communication graph; formation as consensus on offsets.	Convergence under connectivity conditions; robustness to link changes and delays analysed.	Ours is centralized in one process and has no communication model.
HU-MA'AT group move (this document)	Identical arena-frame displacement for every member, admitted only if settled, spaced ≥ 0.88 m, in bounds, and clear of stationary non-members by 1.0 m.	Properties of Section 7: shape preserved in the setpoint model; all-or-nothing admission.	—

Where ours is weaker. No inter-member coupling during the move (shape is kept only because the controllers happen to behave alike: measured shape change ≤ 0.012 m in the swarm test), no analysis of transient spacing, dependence on global positioning and on a single central process, and no avoidance of moving non-members beyond the mode rule. **Where it is adequate.** Short (≤ 2 m) translations of a settled group in an open simulated arena, where the transparency of a single displacement vector and the strict admission checks matter more than tracking performance.

10. Relation to other algorithms

Selected by Algorithm 3 when a move addresses more than one aircraft; uses the arrival rule of Algorithm 9 and the watchdog of Algorithm 7. Algorithm 11 uses group moves (`dx`, `dy`) for sector offsets and survey sweeps, checked through Algorithm 10.

11. References

1. M. A. Lewis and K.-H. Tan, “High precision formation control of mobile robots using virtual structures,” *Autonomous Robots*, vol. 4, no. 4, pp. 387–403, 1997, doi:10.1023/A:1008814708459. `https://doi.org/10.1023/A:1008814708459`
2. J. P. Desai, J. P. Ostrowski and V. Kumar, “Modeling and control of formations of nonholonomic mobile robots,” *IEEE Transactions on Robotics and Automation*, vol. 17, no. 6, pp. 905–908, 2001, doi:10.1109/70.976023. `https://repository.upenn.edu/meam_papers/243`
3. R. Olfati-Saber, J. A. Fax and R. M. Murray, “Consensus and cooperation in networked multi-agent systems,” *Proceedings of the IEEE*, vol. 95, no. 1, pp. 215–233, 2007, doi:10.1109/JPROC.2006.887293. `https://doi.org/10.1109/JPROC.2006.887293`
4. K.-K. Oh, M.-C. Park and H.-S. Ahn, “A survey of multi-agent formation control,” *Automatica*, vol. 53, pp. 424–440, 2015, doi:10.1016/j.automatica.2014.10.022. `https://doi.org/10.1016/j.automatica.2014.10.022`

This specification describes the behaviour of the source revision identified above. Line-level references use `file:function`. Where the code and its own comments disagree, the code is described. Simulation-only system: no statement here is a claim about flight hardware.